

Measuring Fluid Viscosities by Simulating Brownian Motion

Kai J. Rogers

Abstract— In this project, I developed a two-dimensional molecular dynamics simulation of a hard-sphere gas and extended it to study Brownian motion. The aim of this simulation was to measure the viscosity of the simulated 2D hard sphere gas. I used the Einstein–Stokes relation to convert simulated mean squared displacement (MSD) data into a viscosity value. I introduced a tracer particle into the simulated gas and tracked its displacement over time, extracting the diffusion coefficient from an MSD–time linear fit. I then calculated viscosity using a known value for Avogadro’s number and the simulated temperature obtained via the equipartition theorem. To reduce statistical noise and quantify uncertainty, I averaged the results over multiple simulation runs, and got a fluid viscosity of $(6.3 \pm 0.7) \times 10^6 \text{ Pa}\cdot\text{s}$.

I. INTRODUCTION

The aim of this simulation was to measure the viscosity of the simulated 2D hard sphere gas.

Brownian motion is the random movement of particles suspended in a fluid. It provides a natural connection between microscopic and macroscopic physics. In this project, I simulated Brownian motion by introducing a heavy tracer particle into a gas of lighter particles and tracking its displacement over time. By using the data collected from the Brownian motion, I was able to calculate a viscosity of the simulated fluid.

II. THEORY

Brownian motion refers to the random, jittery movement of a particle suspended in a fluid, caused by countless collisions with surrounding molecules. In classical statistical mechanics, this motion is described using diffusion and thermodynamic principles.

The mean squared displacement (MSD) of a particle undergoing Brownian motion increases linearly with time [2]. In two dimensions, this is expressed as:

$$\langle r^2(t) \rangle = 4Dt \quad (1)$$

where $\langle r^2(t) \rangle$ is the mean squared displacement at time t , and D is the diffusion coefficient.

The Einstein-Stokes equation relates the diffusion coefficient to the viscosity of the surrounding fluid:

$$\eta = \frac{RT}{6\pi D N_A r}$$

(2)

Here, η is the dynamic viscosity of the fluid, R is the universal gas constant, T is the absolute temperature, N_A is Avogadro’s number, and r is the radius of the suspended particle. This equation assumes the tracer particle is spherical and much larger than the fluid molecules.

The equipartition equation relates the average kinetic energy of a particle to temperature:

$$\frac{1}{2} m \langle v^2 \rangle = \frac{1}{2} k_B T \quad (3)$$

where m is the particle mass, $\langle v^2 \rangle$ is the mean squared speed, and k_B is Boltzmann’s constant.

To determine when two spherical particles will collide, their trajectories can be analysed geometrically. Suppose two particles have initial positions $\vec{r}_1$ and $\vec{r}_2$, velocities $\vec{v}_1$ and $\vec{v}_2$, and r_1 and r_2 . A collision occurs when the distance between their centres equals the sum of their radii:

$$|\vec{r}_1 + \vec{v}_1 t - (\vec{r}_2 + \vec{v}_2 t)| = r_1 + r_2 \quad (4)$$

Expanding this leads to a quadratic in t .

III. EXPERIMENTAL METHOD

The simulation was built in stages following the project script. I implemented a `Ball` class to represent particles as hard spheres, and a `Container` class, which inherited from `Ball`. This included mass, position, velocity, and radius attributes.

Collisions were handled using 3 main methods, `time_to_collision()`, `move()`, and `collide()`. To find the time to collision between 2 balls, I solved (4). I returned the smallest positive root if it existed, or `None` otherwise. The equation is slightly different for when a ball collides with the container, the last $+$ sign in (4) must be changed to a $-$ sign as the balls are inside each other. I used `isinstance()` to differentiate between ball-ball collisions and ball-container collisions. When the time to next collision was found, the `move()` method would move the balls to the position they should be at that time. I then conducted the collision using `collide()`. I updated the velocities using conservation of momentum and kinetic energy for elastic collisions. For two balls, I calculated the unit normal vector

along the line connecting their centres and applied the 1D elastic collision formula along this axis. The velocity components parallel to unit normal vector were exchanged according to their masses, while perpendicular components were unchanged. For ball–container collisions, I reflected the ball’s velocity about the normal to the container wall and updated the container’s total momentum change to track pressure.

I verified the collision detection and elastic response by constructing a `SingleBallSimulation` and confirming expected motion within the container.

To simulate a gas, I extended this to a `MultiBallSimulation` which distributed multiple particles with randomised directions in rings, allowing for more realistic gas behaviour. The simulation progressed through repeated collision detection and resolution using precise time-of-collision calculations. I confirmed conservation of energy and momentum by calculating the ratio of the values between time t and when $t = 0$. As can be seen on Fig.3, Fig.4, and Fig.5, the ratio is constant with time which confirms the conservation of these values. I also ensured that no particles escaped or overlapped using histograms of radial position and inter-particle distance. As can be seen in Fig.1 and Fig.2, there are no balls closer to each other than the sum of their radii, and there are no balls further away from the centre of the container than the radius of the container take away the radius of the balls.

To study Brownian motion, I created a subclass called `BrownianSimulation`, adding a large tracer particle to the gas. This object had a greater mass than the background particles and was tracked over time. The simulation recorded its position at each step, which I used to calculate the mean squared displacement (MSD). This MSD value was used to calculate the diffusion coefficient D using equation 1.

The background particles were initialised with radius $r = 1$, mass $m = 1$, and speed $v = 10$ in random directions, distributed using concentric rings to ensure a uniform spatial arrangement without overlap. These values were chosen as small radius and mass allowed a large number of particles to move freely in the container, while a speed of 10 ensured sufficiently energetic collisions to maintain a realistic gas-like state.

The container had a radius of 10 simulation units and a mass of 10^7 , effectively rendering it stationary while still tracking momentum transfer for pressure calculations. The tracer particle used in the `BrownianSimulation` class had a radius of 2 and a mass of 100. This gave it significantly more inertia than the background particles, allowing it to undergo Brownian motion rather than being overly influenced by individual collisions. These parameter choices ensured that the tracer experienced many small perturbations, consistent with the assumptions underlying the Einstein–Stokes relation.

To improve the reliability of the MSD-based diffusion coefficient, I applied a burn-in period by discarding the first 50 collisions of each simulation before fitting the linear region of the MSD vs time curve. This is because without the burn-in period, the MSD curve inhibited visible curvature due

to the system not yet reaching thermal equilibrium. This curvature can be seen on fig.14. To justify this choice, I compared the raw MSD data with and without the burn-in. After introducing the burn-in, the MSD increased more linearly with time and the extracted slopes were more stable across runs. This confirmed that allowing the tracer to equilibrate before measurement reduces systematic error and improves the validity of applying the Einstein–Stokes relation.

Each simulation was run for 1000 collisions, with the first 500 discarded as a burn-in period. This allowed the system to reach thermodynamic equilibrium before any data were used to extract physical quantities. The choice of 500 was justified by observing that the MSD curve exhibited non-linear behaviour at early times, gradually becoming linear only after sufficient collisions. This indicates that the tracer had not yet fully thermalised.

I also added physical unit scaling: one simulation unit was treated as one micron in real length, allowing me to convert velocities and positions into SI units. The background temperature was computed using the equipartition theorem, which is equation 3, and the viscosity was then calculated from the MSD slope using the Einstein–Stokes relation, or equation 2. This allowed me to calculate the viscosity of my fluid.

IV. RESULTS

The mean squared displacement (MSD) of the tracer particle was computed across 10 independent simulation runs, each with 500 collisions. A burn-in period of the first 50 collisions was discarded to ensure thermal equilibrium. The MSD curve was observed to grow linearly with time post burn-in, as shown in Fig. 6, validating the diffusive regime.

Linear regression of the MSD–time data yielded a diffusion coefficient of $D = (5.1 \pm 0.6) \times 10^{13}$

After applying the spatial scaling (1 sim unit = 1 μm), and converting the simulated temperature into Kelvin via equation (3), the Einstein–Stokes relation gave a final estimate of the fluid viscosity as $\eta = (6.3 \pm 0.7) \times 10^6 \text{ Pa}\cdot\text{s}$.

V. CONCLUSION

I developed a 2D hard-sphere gas simulation and extended it to model Brownian motion using a tracer particle. By analysing the mean squared displacement and applying the Einstein–Stokes relation, I estimated the viscosity of the simulated fluid. Averaging over multiple runs and applying a burn-in period ensured reliable results.

Although the absolute viscosity value was not physically realistic, the method correctly captured the expected relationship between diffusion, temperature, and viscosity. As a next step, this framework could be adapted to model real fluids by tuning particle properties and scaling more carefully, with the aim of producing physically accurate viscosity estimates.

REFERENCES

[1] Paterson, C., Kingham, R and Richards, A., 2021. Year 2 Computing Projects. [Worksheet] created Oct 2013, revised Oct 2020 & Oct 2021.

[2] A. Einstein, "On the motion of small particles suspended in liquids at rest required by the molecular-kinetic theory of heat," *Annalen der Physik*, vol. 17, pp. 549–560, 1905.

APPENDIX

Axis in graphs 1 – 13 are in simulation units.

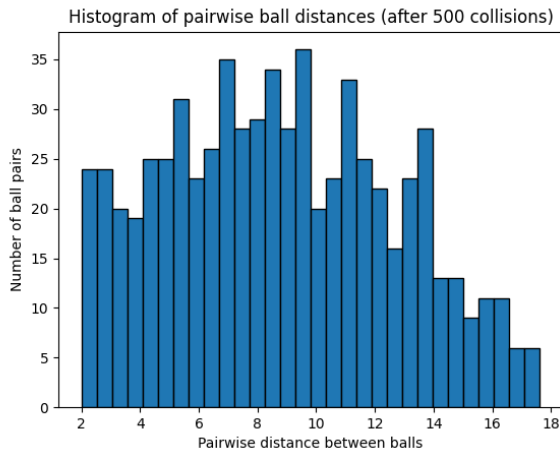


Fig.1. Histogram showing the different ball distances between balls. Notice how there are not ball pairs with a distance greater than 18 or less than 2.

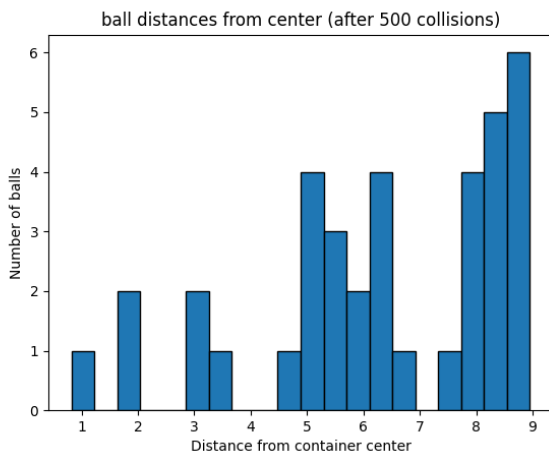


Fig.2. Histogram showing the different distances the balls have from the centre of the container. Notice how there are

none greater than 9, (container radius = 10).

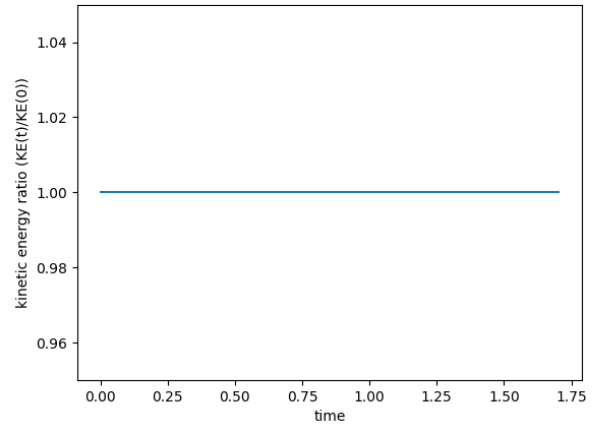


Fig.3. Graph showing the Kinetic energy ratio, the Kinetic energy at time t / the kinetic energy at time 0. Notice how the ratio is equal to 1 for all t .

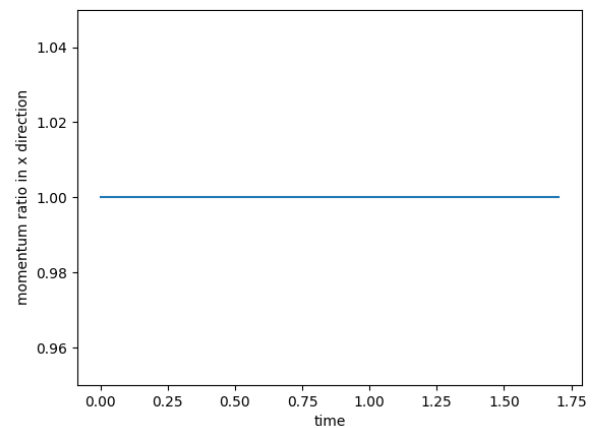


Fig.4. Graph showing the ratio of the x component of the momentum, the momentum at time t / the momentum at time 0. Notice how the ratio is equal to 1 for all t .

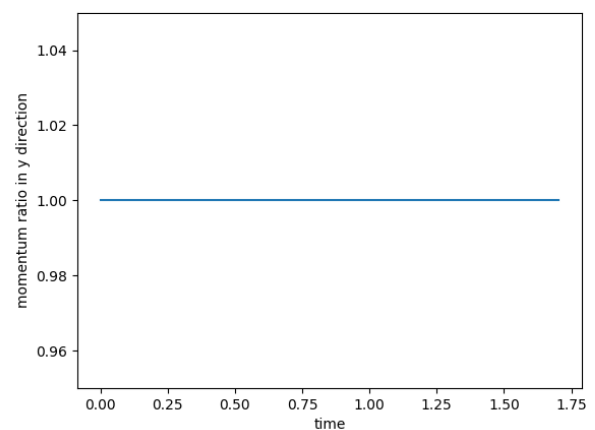


Fig.5. Graph showing the ratio of the y component of the momentum, the momentum at time t / the momentum at time 0. Notice how the ratio is equal to 1 for all t .

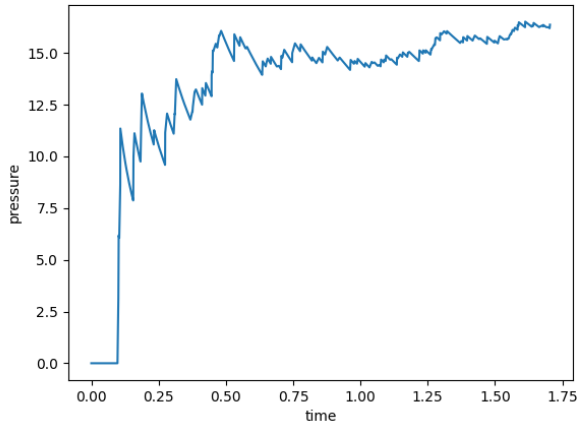


Fig.6. Graph showing the pressure of the system against time. Notice how it takes time to reach equilibrium.

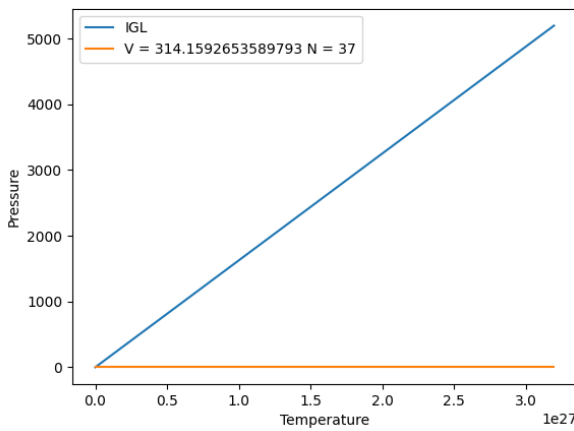


Fig.7. This plot shows the simulated pressure as a function of the temperature obtained from equipartition, alongside the ideal-gas prediction. In simulation units the measured pressures are much smaller than the IGL values, so the data points appear close to zero on this scale.

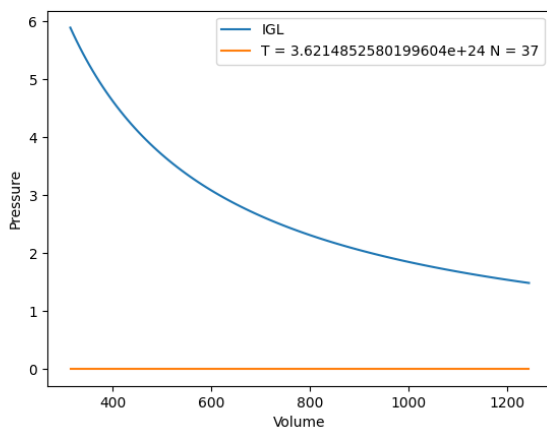


Fig.8. This plot shows the simulated pressure as a function of container volume, compared with the ideal gas law prediction. Because the IGL line is several orders of magnitude larger than the simulated pressures, the simulation points lie near zero on this scale.

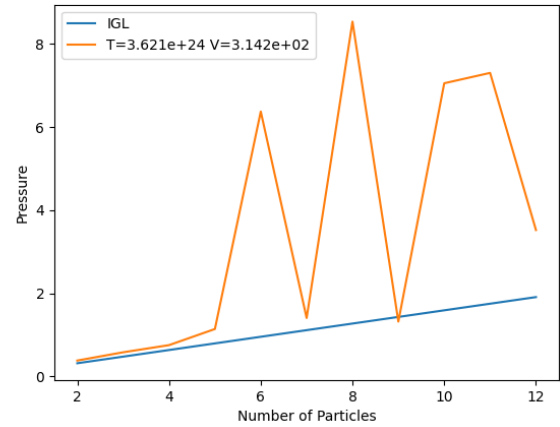


Fig.9. This plot shows how the simulated pressure varies with the number of balls N in the container, with T and V held fixed, alongside the ideal-gas prediction.

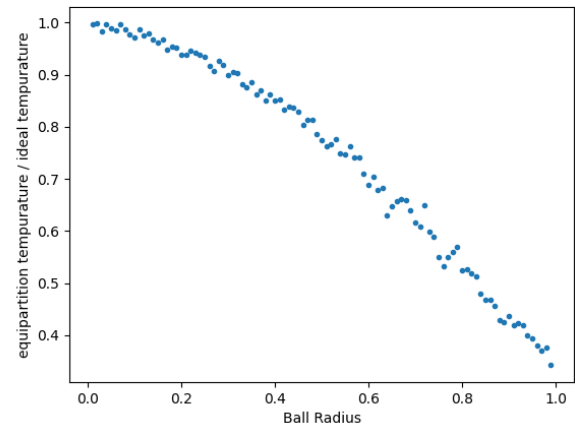


Fig.10. This graph shows the ratio of the temperature obtained from equipartition to the temperature inferred from the ideal gas law as a function of ball radius. For small radii the ratio is close to 1, but it systematically deviates as the balls become larger, quantifying how excluded volume effects cause departures from ideal-gas behaviour.

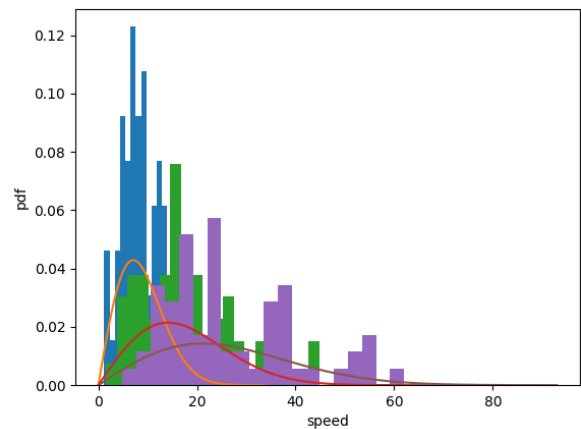


Fig.11. This figure shows a histogram of the simulated particle speeds in equilibrium, with the corresponding Maxwell-Boltzmann distribution overlaid using the simulation's mass and temperature. The histogram follows the theoretical curve, indicating that the simulation reproduces the expected Maxwell-Boltzmann speed

distribution.

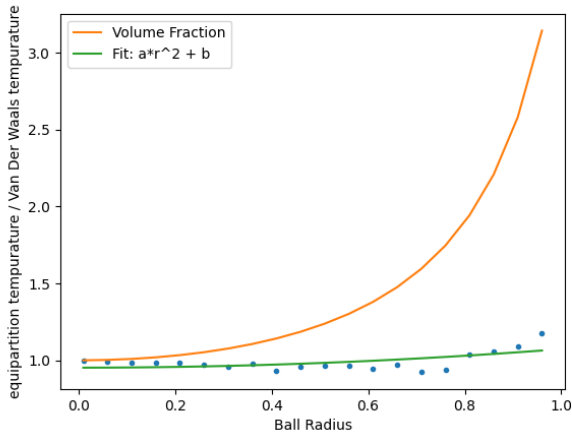


Fig.12. This plot shows the temperature ratio $T_{\text{equipartition}}/T_{\text{ideal}}$ as a function of ball radius, together with a fitted curve and the Van der Waals volume fraction prediction based on the geometric approximation. The fitted curve tracks the simulation data well, while the simple geometric volume fraction begins to deviate at larger radii.

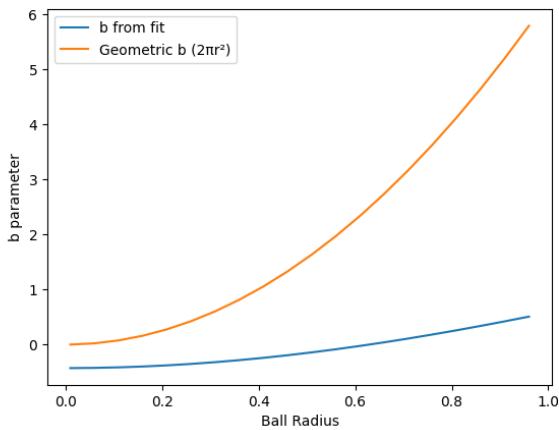


Fig.13. This figure shows the effective Van der Waals b parameter extracted from fitting the simulation data as a function of ball radius, compared with the trivial geometric estimate of b . The two curves have similar trends but do not coincide exactly.

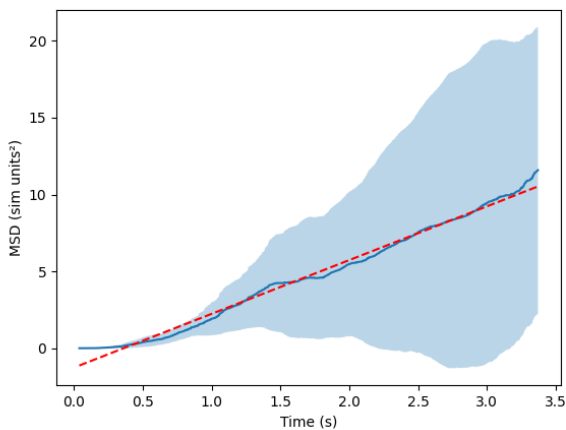


Fig.14. The MSD plotted against time. The curvature at lower times can be seen.